Team Name: *Team # DSCPP-III-2026-T092*	*LifeLink*
Team member 1 (Team Lead)	*Kurupath, Shreeyash Raajendran – 2510370255* *2510370255.geu.ac.in* 
Team member 2	*Sinha, Aman – 2510010550* *myamansinha25@gmail.com* 
Team member 3	*Rawat, Himanshu – 2510010570* *himanshurwt24@gmail.com* 
Team member 4	*Mukherjee, Swastika – 2510370245* *srimi0804123@gmail.com* 

PHASE-II PROJECT PROGRESS DESCRIPTION (35 pts)

Project Abstract

LifeLink will be a Blood Donation Matching Software which will help people to find the suitable Blood Unit for Blood Donation. In emergency situations, it's a good idea to have a donor. LifeLink aims at minimizing time of finding the proper donor in cases of blood emergency. Using this software, you can add a different requirement to: a person, such as blood type match, eligibility, marital status and more. This C++ web-based software application is designed to streamline and secure the blood donation and matching process within blood banks, making it much easier for patients and their families to find suitable donors quickly. Built using the Crow web framework, SQLite for reliable data storage, and JSON for seamless data sharing, the system relies on efficient data structures to optimize donor selection. Specifically, it uses hash maps (`std::unordered_map`) to instantly categorize and look up donors by blood type, a min-heap to prioritize the best candidates based on critical factors like availability and travel time to the hospital, and queues to manage incoming donors who share the same priority level. Ultimately, the project modernizes the entire workflow, ensuring a faster, safer, and more efficient experience for both blood banks and those in urgent need of care. Donation process - methodical using programming, OOP concepts and data structures.

Updated Project Approach and Architecture

The central theme in Phase 1 of the project design was the overall concept of an emergency blood donor matching system – donor management, blood compatibility (including eligibility of donor screening), donor Quality, importance, and letters from donors. In the Coding Phase, the project was broken down into various modules; such that The modules could be used separately in the system, implemented in C++. data layer: Donor module is responsible for handling and keeping data related to donors who are recorded in this system; and Their blood type, where they live, and their medical history. The entire strainer watch list is utilized to retain the rules for Deciding on suitable blood groups. This project combines extract, transform, load and store + retrieve data in a Database. module with sqlite. Logic layer: Blood Bank module contains the information of blood donors, through an unordered map It processes these according to their blood type, organized in this way. All checks of compatibilities are made in the "matching algorithm". eligibility. Next, those suitable donors are put in into a priority queue (min-heap) and are ranked. Links to interface layer: Crow framework built expressions in C++ for the web server.Links to interface layer: Crow framework for creating the web server and expressions were written in C++. nlohmann/json library is used to send and receive the JSON data will be transferred between the server and client. Frontend will consist of HTML, CSS and JavaScript, and will display pages for donors and blood requesters. The users' requests will be matched based on the server and the server will perform the appropriate logic and database operations. operations. All modules will be backend communication; utilizing the C++ functions and linking them as shared objects; frontend using. HTTP request / JSON response will be used to communicate with the server. We are implementing The matching core logic and data structures written in C++. This is for the purpose of the project and so we want to clarify for anyone that has a problem with this clearly and concisely. functions of them. Answering forms of registration and choosing hospitals, eligibility questionnaires and donor ratings will all be As they develop added to the features.

Tasks Completed

Task Completed	Team Member
Worked on the overall system architecture, module planning, and development of the C++ web server using the Crow framework. Verified that the server runs successfully and responds to HTTP requests.	Shreeyash Rajendran Kurupath
Assisted with backend development and integration of the system components for the LifeLink donor matching system.	Aman Sinha
Worked on the backend structure, donor data models, and planning of the blood donor matching workflow.	Himanshu Rawat
This includes integrated SQLite and nlohmann/json with the Actually, C++ project and tested the database connection for the project. and JSON processing. Ensured that the preparation of also worked on. Collect sample donor data (including AI-generated test), To verify the function of the system and check data, Whether the donor matching produces the expected results.	Swastika Mukherjee

Challenges/Roadblocks

It took a bit of work to set up the development environment, including C++ with MSVC, CMake, Crow, SQLite, and nlohmann/json. During development, each library and component was systematically integrated and tested to ensure seamless compatibility. Because blood matching relies on strict, non-negotiable compatibility rules, the system incorporates precise logic that was thoroughly evaluated to accurately identify potential matches. However, finding an acceptable donor is only the first step, as not every registered individual is available or eligible for donation at any given moment. To ensure safety and accuracy, we implemented a robust filtering system that checks health requirements, current availability, and other critical variables before a donor is officially added to the matching list. The system also should prioritize them with the considerations like distance from selected hospital, availability, etc. The data structures are going to be used to store and efficiently rank the candidates.
The project stores its data in SQLite, and runs its web server using Crow. Linking database operations to the proper logic is a challenge as donor data must be saved, retrieved and shared between components without any discrepancy. Library testing has been successful so far, and will need to be fully validated upon integration.

Tasks Pending

Task Pending	Team Member (to complete the task)
Complete blood group compatibility checking and donor eligibility filtering.	Himanshu Rawat
Use Min-Heap to implement donor ranking by distance and by donation history.	Aman Sinha
Set up sample donor data and test it to ensure matching for various blood groups and conditions.	Swastika Mukherjee
Develop a Hash Map based donor registry and include Donor, BloodBank, and MatchRequest classes with the matching logic.	Shreeyash Rajendran Kurupath
Final test, record all the changes made and errors solved, complete documentation and show.	All Team Members

Project Outcome/Deliverables

Our project is to create a system for emergency blood donor matching known as **Emergency Blood Donor Matching System** which helps to find potential blood donors for the person when he needs emergency blood. Using this **Emergency Blood Donor Matching System**, users (i.e., those requiring blood) will have access to a search tool based on their specific needs (blood type, geographic area, etc.), which will then allow them to find possible donors from all over the world.
This project will provide an easy and simple interface that can be easily used by both the patient / relative looking for blood, as well as the blood donor. It is designed to facilitate vital information regarding whether The donor is eligible to donate blood and whether he/she is available presently or not. Summarily speaking, the key result (outcome) of this project will be creation of a blood donor matching system in emergencies, which helps in saving time for both parties (i.e., blood donor and the recipient). Overall this project will demonstrate how programming and technology can help in addressing many daily problems.

Progress Overview

The implementation for the backend is 70-80 percent complete. A C++ web server using the Crow library.Crow based web server in C++. Initial integration testing of SQLite and nlohmann/json is complete while is working fine. The entire system The form and the structure of modules have also been determined. The other tasks which need to be performed Blood Group compatibility, filtering donor eligibility, donor registration etc. part of the attributes that hospitals should have on their computers. Implement using Hash Map and ranking donors by distance with Min Heap based on donation history. Further, testing of system needs to be done using sample data of donors to check matching. The front end, as well as To be designed, the complete user flow, on which the multer depends. The work is progressing nicely so far as the environment is setup and the web, In terms of the initial setup of the server configuration, and initial database integration. However, more work needs to be done in terms of core logic and user interface functionalities

Codebase Information

Repository Link:
https://github.com/SKLynIx7/LifeLink.git

Branch: main

Important Commits:

- Initial project setup and CMake configuration.
- C++ web server setup using the Crow framework.
- Integration of SQLite and nlohmann/json.
- Testing of the web server and database connection.

Testing and Validation Status

Test Type	**Status (Pass/Fail)**	**Notes**
C++ project build and environment configuration	Pass	Project builds successfully using MSVC and CMake.
Crow web server test	Pass	Verified that the server runs and responds to HTTP requests.
SQLite database connection test	Pass	Successfully opened a test database connection.
nlohmann/json library test	Pass	Verified JSON creation and processing.
Integration test for Crow, SQLite, and JSON	Pass	Initial infrastructure integration was tested successfully.
Blood group compatibility and donor eligibility tests	Pending	Will be tested after the matching logic is implemented.
Min-Heap donor ranking test	Pending	Ranking based on distance and donation history remains to be implemented and validated.
Complete donor matching workflow test	Pending	End-to-end testing will be performed after the matching modules are ready.

Deliverables Progress

C++ web server using Crow = Completed and tested.
Integration of SQLite and nlohmann/json = First tests are done.
Architecture of the system and modules planning = completed.
In progress = Donor registry and data model.
Blood group compatibility, donor eligibility and Min-heap based donor ranking = Pending.
Data Preparation and Testing of samples for donors is underway
Frontend and full user workflow = In Progress.
documents: Pending